VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Aaryan Maurya	24BAI10259
Slot	**Date of Submission**
A13	20/03/2026

1. Introduction

Most of the software that powers the modern world — the servers that run websites, the tools that developers use every day, the operating systems inside supercomputers and smartphones — was not built by a single company and sold for profit. It was built openly, shared freely, and improved collectively. That is the essence of open source.

This project asks you to stop and think about that.

You will choose one real open-source software project — something you can download, read about, and interact with on Linux — and conduct a structured audit of it. Not just 'what does it do,' but why it exists, who built it, what freedoms its license actually gives you, what philosophical values it represents, and how it has shaped the world around it.

Alongside the written report, you will write five shell scripts that demonstrate practical Linux skills. These are not meant to be impressive programs — they are meant to show that you understand how open-source tools work at the command line, and how automation connects to the philosophy of sharing and transparency.

Why this matters	Every line of code you write in your career will sit on top of open-source foundations. Understanding the philosophy behind that is not optional — it is part of being a professional.

2. Choosing Your Software

Pick one open-source project from the list below, or propose your own (subject to instructor approval). Your choice will be the subject of your entire report and all five shell scripts.

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	GPL v2	*The foundation everything else runs on*
Apache HTTP Server	Web Server	Apache 2.0	*Powers ~30% of all websites globally*
MySQL	Database	GPLv2/Commercial	*A license with a dual-model story to tell*
Firefox	Web Browser	MPL 2.0	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	LGPL / GPL	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	MPL 2.0	*Born from a community fork — a real lesson*
Python	Programming Language	PSF License	*A language shaped entirely by community*

Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Your choice	Write your chosen software here before submission: **Git**

3. Project Report Structure

Your report should be 12 to 16 pages long. Each section below maps to a unit in the course. Do not pad with screenshots — every page should contain original thinking and analysis written in your own words.

Part A — Origin and Philosophy (Units 1 & 2)

This is the most important section of the report. Allocate at least four pages to it.

A1. The problem this software was created to solve

Git was created by Linus Torvalds in 2005 during a crucial time in the development of the Linux kernel. Before Git, the Linux community used a proprietary version control system called BitKeeper. This tool was free for developers but not open source. Relying on closed software led to problems for managing one of the world's most important open-source projects.

Things became complicated when the relationship between Linux developers and BitKeeper's company soured. Access was revoked, leaving the Linux community without a reliable way to manage contributions from thousands of developers around the world. The alternatives at the time were either too slow, lacked distributed features, or weren't built to handle large-scale collaboration well.

Faced with this crisis, Torvalds decided to create a new system from scratch. His goals were clear:

- It should be fast.
- It should support distributed development.
- It should maintain data integrity.
- It should be open source.

Unlike earlier systems, Git was designed so that every developer had a complete copy of the repository, including its history. This setup allowed development to continue independently without depending on a central server. This approach changed how software is developed.

Git wasn't just a technical solution; it was also a response to the risks of relying on proprietary tools. By making Git open source, Torvalds ensured that no single group could control or limit access to it. Today, Git is essential to platforms like GitHub and GitLab and is used by millions of developers worldwide.

A2. The license — what it actually says

Git is released under the GNU General Public License version 2 (GPL v2). This license is one of the most important in the open-source world because it guarantees user freedoms while also enforcing certain responsibilities.

The Four Freedoms of Free Software

Defined by the GNU Project, the four freedoms are:

- Freedom to run the program for any purpose
- Freedom to study how the program works
- Freedom to modify the program
- Freedom to distribute copies (modified or not)

Git's GPL license ensures that all these freedoms are granted.

Can a Company Modify and Sell Git Without Sharing Changes?

No, not completely. Under GPL v2:

A company can sell a modified version of Git

But they must release the source code of their modifications

This is called a copyleft license. It prevents companies from taking open-source work, modifying it, and turning it into closed software.

Feature	GPL	MIT
Code must remain open	Yes	No
Commercial use	Allowed	Allowed
Modification sharing	Required	Optional

If I built my own software, I would choose GPL if I wanted to ensure openness and MIT if I wanted maximum adoption, even in proprietary systems.

Free as in "Free Beer" vs "Free Speech"

- Free beer → no cost
- Free speech → freedom to modify and share

Git is free in both senses:

- You can download it without paying
- You can modify and redistribute it freely

A3. The ethics of open source

Open source is not just about code; it reflects a philosophy about sharing knowledge.

Should All Software Be Open Source?

Arguments FOR:

- Transparency improves security.
- It encourages collaboration.
- It reduces monopoly power.

Arguments AGAINST:

- Companies need profit incentives.
- Some software requires secrecy, such as security systems.
- Open source may lack dedicated support.

Is It Ethical for Companies to Profit from Open Source?

Companies like:

- Red Hat
- Google

build profitable services using open-source tools like Linux and Git. This raises ethical questions. They benefit from community work, but they also contribute back in many cases. In my view, it is ethical only if companies give back through code, funding, or community support.

"Standing on the Shoulders of Giants"

This phrase means innovation builds on earlier work. In software, Git itself builds on earlier ideas. Modern tools build on Git. Open source accelerates innovation instead of limiting it.

There are no correct answers here. We are grading the depth of your thinking, not your conclusion.

Part B — Linux Footprint (Unit 2)

Installation Method

Git is easily installed using:

```
sudo apt install git
```

This shows how open-source tools are distributed through package managers.

Key Directories

After installation:

- /usr/bin/git → main executable
- /etc/gitconfig → system-wide config
- ~/.gitconfig → user config
- /var/log → logs (if applicable)

User and Group

Git runs under the **current user**, not as root.

This is important because:

- Reduces security risks
- Prevents system-wide damage

Service Management

Git is not a background service like a web server. It is a command-line tool.

Example:

```
git --version
git init
```

Update Model

Updates come via:

```
sudo apt update
sudo apt upgrade
```

```bash
#!/bin/bash
# ============================================================
# Script 1: System Identity Report
# Author: Aaryan Maurya | Roll: 24BAI10259
# Course: Open Source Software | Software Choice: Git
# Description: Displays a welcome screen with system info
# ============================================================

# --- Variables ---
STUDENT_NAME="Aaryan Maurya"
SOFTWARE_CHOICE="Git"
LICENSE="GPL v2"

# --- Gather system information using command substitution ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date '+%d %B %Y %H:%M:%S')
DISTRO=$(lsb_release -d | cut -f2)

# --- Display formatted output ---
echo "==================================================="
echo "       Open Source Audit — $STUDENT_NAME"
echo "       Chosen Software: $SOFTWARE_CHOICE"
echo "==================================================="
echo ""
echo "  System Information"
echo "  ------------------"
echo "  Distribution : $DISTRO"
echo "  Kernel       : $KERNEL"
echo "  Logged in as : $USER_NAME"
echo "  Home Dir     : $HOME_DIR"
echo "  Uptime       : $UPTIME"
echo "  Date & Time  : $CURRENT_DATE"
echo ""
echo "  License Info"
echo "  ------------"
echo "  This system runs Linux, which is licensed under"
echo "  the $LICENSE (GNU General Public License v2)."
echo "  Git, our chosen software, is also under $LICENSE."
echo "  This means you are free to use, study, modify,"
echo "  and redistribute both Linux and Git freely."
echo ""
echo "==================================================="
echo "  'In real open source, you have the right to"
echo "   control your own destiny.' — Linus Torvalds"
echo "==================================================="
```

Security patches are contributed by the global community and distributed quickly.

Part C — The FOSS Ecosystem (Units 3 & 4)

Dependencies

Git depends on:

- C programming language
- Standard libraries
- Shell utilities

What Git Enables

Git has enabled:

- GitHub
- GitLab
- Open-source collaboration

Without Git, modern development workflows would not exist.

Role in Web / Development Stack

Git is not part of LAMP directly, but it supports:

- Code deployment
- Version tracking
- Collaboration

Community

Git has:

- Mailing lists
- Global contributors
- Maintainers led by Torvalds

Governance is open but guided by experienced maintainers.

Part D — Open Source vs Proprietary (Critical Analysis)

Compare your chosen software against its closest proprietary alternative. Be specific and honest — do not assume open source is always better.

Dimension	Open Source (Git)	Proprietary Alternative
Cost	Completely free to use, modify, and distribute	Often requires licensing fees or subscriptions
Security (who can audit the code?)	Source code is publicly available and can be audited by anyone	Source code is closed; users must trust the company

Support and reliability	Community-driven support through forums, documentation, and contributors	Dedicated professional support with service-level agreements
Freedom to modify	Highly customizable and adaptable to different workflows	Limited customization based on company restrictions

Community vs corporate control	Controlled by a global community, no single owner	Controlled by a company with full authority over features and changes
Your overall verdict	Git is more accessible, transparent, flexible, and gives users full control. While proprietary tools may offer structured support, Git's open nature and strong community make it the better overall choice for most real-world scenarios.	Proprietary tools can be useful in enterprise environments requiring guaranteed support, but they limit user freedom and transparency.

Fill in the table as part of your report, then write a two-paragraph conclusion: given everything you have learned, would you recommend your chosen software for a real-world deployment? Would you contribute to it?

Git is one of the most influential open-source tools in modern software development, showing how a community-driven approach can produce software that is both powerful and widely trusted. Developed by Linus Torvalds, Git emerged out of necessity but quickly evolved into a global standard for version control. Its transparency is one of its greatest strengths, as the open availability of its source code allows developers to inspect, modify, and improve it freely. This openness builds trust and enables faster identification and resolution of security issues, unlike proprietary tools that rely on closed systems and company-controlled updates.

Another key advantage of Git is its flexibility and distributed architecture, which allows users to work independently while maintaining complete copies of project repositories. This makes it highly reliable and suitable for both small and large-scale development. While proprietary tools may offer structured support and enterprise features, they often limit user control and customization. Overall, Git provides a more balanced and empowering solution by combining technical efficiency with the core values of open-source development. It is highly recommended for real-world use, not only for its capabilities but also for the opportunity it offers to participate in a collaborative and innovative global ecosystem.

4. Shell Script Tasks

You will write five shell scripts. Each script must be submitted as a plain .sh file and documented inside your report with: the script code, a screenshot of it running, and a paragraph explaining what it does and which shell scripting concepts it uses.

Important	Scripts must run on a real Linux system. They must include comments explaining each section. Uncommented scripts will lose marks.

Script 1 — System Identity Report (Unit 1 & 2)

Write a script that introduces the Linux system like a welcome screen. It should display:

- The Linux distribution name and kernel version
- The current logged-in user and their home directory
- System uptime and current date/time
- A message stating which open-source license covers the OS

```bash
#!/bin/bash
# ============================================================
# Script 1: System Identity Report
# Author: Aaryan Maurya | Roll: 24BAI10259
# Course: Open Source Software | Software Choice: Git
# Description: Displays a welcome screen with system info
# ============================================================

# --- Variables ---
STUDENT_NAME="Aaryan Maurya"
SOFTWARE_CHOICE="Git"
LICENSE="GPL v2"

# --- Gather system information using command substitution ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date '+%d %B %Y %H:%M:%S')
DISTRO=$(lsb_release -d | cut -f2)

# --- Display formatted output ---
echo "==================================================="
echo "       Open Source Audit — $STUDENT_NAME"
echo "       Chosen Software: $SOFTWARE_CHOICE"
echo "==================================================="
echo ""
echo "  System Information"
echo "  ------------------"
echo "  Distribution : $DISTRO"
echo "  Kernel       : $KERNEL"
echo "  Logged in as : $USER_NAME"
echo "  Home Dir     : $HOME_DIR"
echo "  Uptime       : $UPTIME"
echo "  Date & Time  : $CURRENT_DATE"
echo ""
echo "  License Info"
echo "  ------------"
echo "  This system runs Linux, which is licensed under"
echo "  the $LICENSE (GNU General Public License v2)."
echo "  Git, our chosen software, is also under $LICENSE."
echo "  This means you are free to use, study, modify,"
echo "  and redistribute both Linux and Git freely."
echo ""
echo "==================================================="
echo "  'In real open source, you have the right to"
echo "   control your own destiny.' — Linus Torvalds"
echo "==================================================="
```

Concepts to use: variables, echo, command substitution ($()), basic output formatting.

Output:

```
warpy@TheRealSlimShady:/mnt/c/Computer/oss-audit-24BAI10259$ sudo bash script1_system_identity.sh
[sudo] password for warpy:
==================================================
       Open Source Audit — Aaryan Maurya
       Chosen Software: Git
==================================================

  System Information
  ------------------
  Distribution : Ubuntu 24.04.4 LTS
  Kernel       : 6.6.87.2-microsoft-standard-WSL2
  Logged in as : root
  Home Dir     : /root
  Uptime       : up 20 minutes
  Date & Time  : 25 March 2026 18:46:49

  License Info
  ------------
  This system runs Linux, which is licensed under
  the GPL v2 (GNU General Public License v2).
  Git, our chosen software, is also under GPL v2.
  This means you are free to use, study, modify,
  and redistribute both Linux and Git freely.

==================================================
  'In real open source, you have the right to
   control your own destiny.' — Linus Torvalds
==================================================
```

Script 2 — FOSS Package Inspector (Unit 2)

Write a script that checks whether your chosen software is installed on the system, finds its version, and uses a case statement to print a short description of its purpose based on the package name.

Concepts to use: if-then-else, case statement, rpm -qi or dpkg -l, pipe with grep.

```bash
#!/bin/bash
# ============================================================
# Script 2: FOSS Package Inspector
# Author: Aaryan Maurya | Roll: 24BAI10259
# Course: Open Source Software | Software Choice: Git
# Description: Checks if a FOSS package is installed,
#              shows its details, and prints a philosophy note
# ============================================================

# --- Package to inspect ---
PACKAGE="git"

echo "==================================================="
echo "       FOSS Package Inspector"
echo "==================================================="
echo ""

# --- Check if the package is installed using dpkg (Ubuntu/Debian) ---
if dpkg -l "$PACKAGE" 2>/dev/null | grep -q "^ii"; then
    # Package IS installed — show its details
    echo "  [✓] '$PACKAGE' is installed on this system."
    echo ""
    echo "  Package Details:"
    echo "  ----------------"

    # Extract version number from dpkg output
    VERSION=$(dpkg -l "$PACKAGE" | grep "^ii" | awk '{print $3}')
    echo "  Version  : $VERSION"

    # Show full package info using apt-cache
    apt-cache show "$PACKAGE" 2>/dev/null | grep -E "^(Version|License|Maintainer|Homepage|Description)" | head -10

else
    # Package is NOT installed — prompt user to install
    echo "  [✗] '$PACKAGE' is NOT installed."
    echo ""
    echo "  To install it, run:"
    echo "  sudo apt install $PACKAGE"
fi

echo ""
echo "  Philosophy Notes (by package):"
echo "  ------------------------------"

# --- Case statement: print a philosophy note based on package name ---
case "$PACKAGE" in
    git)
        echo "  Git: Born from necessity — Linus built it in 2 weeks"
        echo "  after a proprietary tool abandoned the Linux project."
        echo "  It proves open-source tools can outperform proprietary ones."
        ;;
    apache2)
        echo "  Apache: The web server that democratised t
        ;;
esac

echo ""
echo "==================================================="
```

Output:

```
warpy@TheRealSlimShady:/mnt/c/Computer/oss-audit-24BAI10259$  sudo bash script2_package_inspector.sh
==================================================
       FOSS Package Inspector
==================================================

  [✓] 'git' is installed on this system.

  Package Details:
  ----------------
  Version  : 1:2.43.0-1ubuntu7.3
Version: 1:2.43.0-1ubuntu7.3
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Homepage: https://git-scm.com/
Description-en: fast, scalable, distributed revision control system
Description-md5: c1f968556452a190fe359bffd151c012
Version: 1:2.43.0-1ubuntu7
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Homepage: https://git-scm.com/
Description-en: fast, scalable, distributed revision control system
Description-md5: c1f968556452a190fe359bffd151c012

  Philosophy Notes (by package):
  ------------------------------
  Git: Born from necessity — Linus built it in 2 weeks
  after a proprietary tool abandoned the Linux project.
  It proves open-source tools can outperform proprietary ones.

==================================================
```

Script 3 — Disk and Permission Auditor (Unit 2)

Write a script that loops through a list of important system directories and reports: how much space each uses, and the owner and permissions of the directory. Use a for loop. Concepts to use: for loop, df, ls -ld, awk or cut to extract fields.

Code structure

```bash
#!/bin/bash
# ============================================================
# Script 3: Disk and Permission Auditor
# Author: Aaryan Maurya | Roll: 24BAI10259
# Course: Open Source Software | Software Choice: Git
# Description: Loops through key system directories and
#              reports their size, owner, and permissions
# ============================================================

# --- List of important system directories to audit ---
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "==================================================="
echo "         Disk and Permission Auditor"
echo "==================================================="
echo ""
echo "  Auditing key system directories..."
echo ""
printf "  %-20s %-25s %-10s\n" "Directory" "Permissions (type/owner/group)" "Size"
printf "  %-20s %-25s %-10s\n" "---------" "-----------------------------" "----"

# --- Loop through each directory in the list ---
for DIR in "${DIRS[@]}"; do

    # Check if the directory actually exists before inspecting
    if [ -d "$DIR" ]; then

        # Extract permissions, owner, and group using ls and awk
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')

        # Get human-readable size using du, suppress permission errors
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        # Print formatted row
        printf "  %-20s %-25s %-10s\n" "$DIR" "$PERMS" "$SIZE"

    else
        # Directory doesn't exist on this system
        printf "  %-20s %s\n" "$DIR" "[ does not exist on this system ]"
    fi

done

echo ""
echo "==================================================="
echo "  Git-Specific Directory Check"
echo "==================================================="
echo ""

# --- Check Git's config directory and binary location ---
GIT_CONFIG="$HOME/.gitconfig"
GIT_BINARY="/usr/bin/git"
GIT_SYSTEM_CONFIG="/etc/gitconfig"

# Check user git config file
if [ -f "$GIT_CONFIG" ]; then
    PERMS=$(ls -l "$GIT_CONFIG" | awk '{print $1, $3, $4}')
    SIZE=$(du -sh "$GIT_CONFIG" 2>/dev/null | cut -f1)
    echo "  [✓] Git user config found: $GIT_CONFIG"
    echo "      Permissions: $PERMS | Size: $SIZE"
else
    echo "  [!] No user Git config found at $GIT_CONFIG"
    echo "      (Run 'git config --global user.name YourName' to create one)"
fi

echo ""

# Check git binary
if [ -f "$GIT_BINARY" ]; then
    PERMS=$(ls -l "$GIT_BINARY" | awk '{print $1, $3, $4}')
    echo "  [✓] Git binary found: $GIT_BINARY"
    echo "      Permissions: $PERMS"
else
    echo "  [✗] Git binary not found at $GIT_BINARY"
fi

echo ""

# Check system-wide git config
if [ -f "$GIT_SYSTEM_CONFIG" ]; then
    echo "  [✓] System-wide Git config found: $GIT_SYSTEM_CONFIG"
else
    echo "  [!] No system-wide Git config at $GIT_SYSTEM_CONFIG"
fi

echo ""
echo "==================================================="
```

Output :

```
warpy@TheRealSlimShady:/mnt/c/Computer/oss-audit-24BAI10259$ sudo bash script3_disk_permission_auditor.sh
==================================================
         Disk and Permission Auditor
==================================================

  Auditing key system directories...

  Directory            Permissions (type/owner/group) Size
  ---------            ------------------------------ ----
  /etc                 drwxr-xr-x root root      4.6M
  /var/log             drwxrwxr-x root syslog    380M
  /home                drwxr-xr-x root root      32K
  /usr/bin             drwxr-xr-x root root      223M
  /tmp                 drwxrwxrwt root root      32K

==================================================
  Git-Specific Directory Check
==================================================

  [!] No user Git config found at /root/.gitconfig
      (Run 'git config --global user.name YourName' to create one)

  [✓] Git binary found: /usr/bin/git
      Permissions: -rwxr-xr-x root root

  [!] No system-wide Git config at /etc/gitconfig

==================================================
```

Script 4 — Log File Analyzer (Units 2 & 5)

Write a script that reads a log file line by line, counts how many lines contain a keyword (like ERROR or WARNING), and prints a summary. Use a while-read loop and an if-then inside it.

Concepts to use: while read loop, if-then, counter variables, command-line arguments ($1).

Code structure:

```bash
#!/bin/bash
# ============================================================
# Script 4: Log File Analyzer
# Author: Aaryan Maurya | Roll: 24BAI10259
# Course: Open Source Software | Software Choice: Git
# Description: Reads a log file, counts keyword occurrences,
#              and shows the last 5 matching lines
# Usage: ./script4_log_analyzer.sh [logfile] [keyword]
# Example: ./script4_log_analyzer.sh /var/log/dpkg.log git
# ============================================================

# --- Accept log file path as first argument ($1) ---
LOGFILE=$1

# --- Accept keyword as second argument, default to 'error' if not given ---
KEYWORD=${2:-"error"}

# --- Counter to track how many matching lines we find ---
COUNT=0

echo "==================================================="
echo "            Log File Analyzer"
echo "==================================================="
echo ""

# --- Validate that a log file argument was provided ---
if [ -z "$LOGFILE" ]; then
    echo "  [!] No log file specified."
    echo "      Usage: $0 <logfile> [keyword]"
    echo "      Example: $0 /var/log/dpkg.log git"
    exit 1
fi

# --- Retry loop: check if file exists, retry up to 3 times ---
RETRIES=0
MAX_RETRIES=3

while [ ! -f "$LOGFILE" ] && [ $RETRIES -lt $MAX_RETRIES ]; do
    echo "  [!] File not found: $LOGFILE"
    RETRIES=$((RETRIES + 1))

    # If we haven't hit max retries, ask for a new path
    if [ $RETRIES -lt $MAX_RETRIES ]; then
        read -p "  Enter a valid log file path (attempt $((RETRIES+1))/$MAX_RETRIES): " LOGFILE
    fi
done

# --- If file still not found after retries, exit with error ---
if [ ! -f "$LOGFILE" ]; then
    echo "  [✗] Could not find a valid log file after $MAX_RETRIES attempts."
    echo "      Tip: Try /var/log/dpkg.log or /var/log/syslog"
    exit 1
fi

# --- Check if the file is empty ---
if [ ! -s "$LOGFILE" ]; then
    echo "  [!] Warning: The file '$LOGFILE' is empty. Nothing to analyze."
    exit 0
fi

echo "  Analyzing  : $LOGFILE"
echo "  Keyword    : '$KEYWORD'"
echo ""
# --- Read the log file line by line using a while-read loop ---
while IFS= read -r LINE; do

    # Check if the current line contains the keyword (case-insensitive)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))   # Increment counter for each match
    fi

done < "$LOGFILE"   # Feed the file into the while loop

# --- Print the summary ---
echo "  Result: Keyword '$KEYWORD' found $COUNT time(s) in:"
echo "  $LOGFILE"
echo ""
```

```
# --- Show last 5 matching lines for context ---
echo "  Last 5 matching lines:"
echo "  ----------------------"
grep -i "$KEYWORD" "$LOGFILE" | tail -5 | while IFS= read -r MATCH; do
    echo "  > $MATCH"
done

echo ""
echo "================================================"
echo "  Tip: For Git-related logs, try:"
echo "  $0 /var/log/dpkg.log git"
echo "================================================"
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

This is the most creative script. Write a script that generates a personalised open source philosophy statement by asking the user three questions interactively, then composing a short paragraph using their answers and saving it to a .txt file.

Concepts to use: read for user input, string concatenation, writing to a file with >, date command, aliases concept demonstrated via a comment.

Code structure:

```bash
#!/bin/bash
# ==========================================================
# Script 5: Open Source Manifesto Generator
# Author: Aaryan Maurya | Roll: 24BAI10259
# Course: Open Source Software | Software Choice: Git
# Description: Asks the user 3 questions interactively and
#              generates a personalised open source manifesto
# ==========================================================

# --- Alias concept (demonstrated via comment and function) ---
# In a real shell session you could set: alias today='date +%d\ %B\ %Y'
# Here we achieve the same with a variable for portability in scripts

# Get today's date in readable format
DATE=$(date '+%d %B %Y')

# Define output filename using the current username
OUTPUT="manifesto_$(whoami).txt"

echo "================================================"
echo "       Open Source Manifesto Generator"
echo "================================================"
echo ""
echo "  Answer three questions to generate your personal"
echo "  open source philosophy statement."
echo ""

# --- Question 1: Ask for a tool they use daily ---
read -p "  1. Name one open-source tool you use every day: " TOOL

# --- Question 2: Ask what freedom means to them ---
read -p "  2. In one word, what does 'freedom' mean to you? " FREEDOM

# --- Question 3: Ask what they would build and share ---
read -p "  3. Name one thing you would build and share freely: " BUILD

echo ""
echo "  Generating your manifesto..."
echo ""

# --- Compose the manifesto by concatenating strings and writing to file ---
# Using > to create/overwrite the file, then >> to append each line

echo "================================================" > "$OUTPUT"
echo "       MY OPEN SOURCE MANIFESTO" >> "$OUTPUT"
echo "       Generated on: $DATE" >> "$OUTPUT"
echo "       By: $(whoami)" >> "$OUTPUT"
echo "================================================" >> "$OUTPUT"
echo "" >> "$OUTPUT"

# Main manifesto paragraph — built from user's answers
echo "  I believe in the power of open source software." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "  Every day, I rely on $TOOL — a tool built not by" >> "$OUTPUT"
echo "  a corporation for profit, but by a community for" >> "$OUTPUT"
echo "  the common good. To me, freedom means $FREEDOM." >> "$OUTPUT"
echo "  That is the same freedom that open source gives" >> "$OUTPUT"
echo "  every developer: the freedom to read, change," >> "$OUTPUT"
echo "  and share the tools they depend on." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "  Inspired by projects like Git — built by Linus" >> "$OUTPUT"
echo "  Torvalds and shared with the world under the" >> "$OUTPUT"
echo "  GPL v2 license — I commit to contributing to" >> "$OUTPUT"
echo "  the open-source community. One day, I will build" >> "$OUTPUT"
echo "  $BUILD and share it freely, so that others may" >> "$OUTPUT"
echo "  build on my work just as I have built on theirs." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "  Standing on the shoulders of giants, I choose" >> "$OUTPUT"
echo "  to be a giant for the next generation." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "================================================" >> "$OUTPUT"
echo "  Signed: $(whoami) | $DATE" >> "$OUTPUT"
echo "================================================" >> "$OUTPUT"

# --- Confirm the file was saved and display it ---
echo "  [✓] Manifesto saved to: $OUTPUT"
echo ""
echo "--- Your Manifesto ---"
echo ""

# Display the saved manifesto
cat "$OUTPUT"
```

Output:

```
==================================================
       MY OPEN SOURCE MANIFESTO
       Generated on: 25 March 2026
       By: root
==================================================

  I believe in the power of open source software.

  Every day, I rely on Linux — a tool built not by
  a corporation for profit, but by a community for
  the common good. To me, freedom means not being bounded.
  That is the same freedom that open source gives
  every developer: the freedom to read, change,
  and share the tools they depend on.

  Inspired by projects like Git — built by Linus
  Torvalds and shared with the world under the
  GPL v2 license — I commit to contributing to
  the open-source community. One day, I will build
  'startup and share it freely, so that others may
  build on my work just as I have built on theirs.
  Standing on the shoulders of giants, I choose
  to be a giant for the next generation.

==================================================
  Signed: root | 25 March 2026
==================================================
```

5. Project Timeline

This timeline is a guide, not a rigid rule. Some students will move faster on the technical parts; others will spend more time on the philosophical sections. Adjust based on your strengths — but do not leave the scripts to the final stretch.

Day	Focus	Tasks	Deliverable
1	**Setup**	Choose software. Research its origin story. Read the license text. Set up your Linux environment (VM or lab system). Install your chosen software.	*Choice confirmed*
2	**Philosophy**	Write Part A1 — the origin story. Take notes on what problem the software solved and for whom.	*Draft A1*
3	**License & Ethics**	Write Part A2 — license analysis. Write Part A3 — ethical reflection. These sections need your own thinking, not summaries.	*Draft A2 + A3*
4	**Linux Footprint**	Install software on Linux. Document directories, user, permissions. Write Part B.	*Draft Part B + screenshots*
5	**Scripts 1 & 2**	Write and test Script 1 (System Identity) and Script 2 (Package Inspector). Add them to your report with explanations.	*Scripts 1 – 2 working*
6	**Scripts 3 & 4**	Write and test Script 3 (Disk Auditor) and Script 4 (Log Analyzer). Test with a real log file.	*Scripts 3 – 4 working*

7	**Ecosystem**	Write Part C — FOSS ecosystem map. Research dependencies and community.	*Draft Part C*
8	**Comparison & Script 5**	Write Part D — comparison table and verdict. Write and test Script 5 (Manifesto Generator).	*Draft Part D + Script 5*
9	**Report polish**	Compile full report. Write introduction and conclusion. Add screenshots. Check page count (12–16 pages).	*Complete draft*
10	**Submission**	Final review. Submit report + 5 .sh files. Be ready to explain any script in a short viva if asked.	*Final submission*

6. Evaluation Rubric

Total marks: 100. The report carries 60 marks and the shell scripts carry 40 marks. There is no viva unless the evaluator suspects the work is not original — in which case any student may be called for a brief oral explanation.

Report (60 marks)

#	Component	Full marks	Passing (50%)	Notes
1	Origin story — depth, original research, storytelling	12	6	*Part A1*
2	License analysis — accuracy and understanding of freedoms	12	6	*Part A2*
3	Ethical reflection — quality of argument and original thinking	10	5	*Part A3*
4	Linux footprint — technical accuracy and screenshots	10	5	*Part B*
5	Ecosystem and LAMP connection	8	4	*Part C*
6	Open source vs proprietary comparison	8	4	*Part D*

Shell Scripts (40 marks — 8 marks each)

Each script is marked on three criteria:

Criterion	Marks	What we look for
Correctness — does it run without errors?	4	*Script executes on Linux and produces expected output*
Concepts — does it use the right shell constructs?	2	*Loops, if-then, case, variables used appropriately for the task*
Comments and documentation	2	*Every non-obvious line has a comment; script purpose is clear*

Academic integrity	All written sections must be in your own words. Code that is copy-pasted from the internet without understanding will be zero-marked. Running a script and explaining what it does in a conversation is considered a passing demonstration of understanding.

7. Reference Resources

Use these as starting points, not as sources to copy from. The report must be in your own words.

On open source philosophy

- GNU Project — 'The Free Software Definition': gnu.org/philosophy/free-sw.html
- Open Source Initiative — 'The Open Source Definition': opensource.org/osd
- Eric S. Raymond — 'The Cathedral and the Bazaar': catb.org/~esr/writings/cathedral-bazaar
- Linus Torvalds — 'Just for Fun' (autobiography, available at most libraries)

On Linux and shell scripting

- The Linux Command Line by William Shotts — freely available at linuxcommand.org
- GNU Bash Manual: gnu.org/software/bash/manual
- Linux man pages: man.cx (online) or type 'man <command>' in your terminal

For finding license texts

- SPDX License List: spdx.org/licenses
- Choose a License: choosealicense.com
- Your software's official repository on GitHub or GitLab — the LICENSE file is always there

A closing thought

Every tool you will use in your career — the editor, the compiler, the server, the database — was shaped by people who chose to build in the open and share their work freely. Understanding why they made that choice, and what it means for the world, is not a footnote to your technical education. It is the foundation of it.

8. Submission Requirements

All submissions are made through the VITyarthi portal. You must submit three things: a public GitHub repository link, a README.md inside the repository, and a project report PDF. All three are compulsory. Incomplete submissions will not be evaluated.

What to submit on the VITyarthi portal

Item	Details	Compulsory?
GitHub repo link	The repository must be public. It must contain all five .sh script files and a README.md. Name the repo: oss-audit-[rollnumber]	**Yes**
README.md	Must be inside the GitHub repository. Must include: student name and roll number, chosen software, description of each script, step-by-step instructions to run each script on Linux, and any dependencies required.	**Yes**

Project report PDF	Upload directly on the VITyarthi portal. Must follow the structure in Section 3. Must be 12 to 16 pages. Must be your own writing.	**Yes**

On the VITyarthi portal submission form, paste the GitHub repository URL, confirm the README is present in the repository, and upload the project report PDF. All three must be submitted together in a single submission — partial submissions will not be accepted.

Evaluation pipeline

Every submission goes through an automated evaluation pipeline before it reaches an evaluator. Be aware of the following checks that run on every submitted report and script set.

Check	What it detects	Consequence
Plagiarism detection	Report text copied from websites, Wikipedia, other students' submissions, or any published source without attribution. Cross-submission similarity between enrolled students is also flagged.	Zero on the full report. Referred to disciplinary committee if similarity exceeds 40%.
AI generation detection	Report sections generated by AI tools such as ChatGPT, Gemini, Claude, or similar. The pipeline checks for linguistic patterns, sentence uniformity, and structural signatures common to AI-generated text.	Flagged submissions are called for a mandatory viva. If the student cannot explain the content, zero marks are awarded.

A note on AI tools: you may use AI to understand concepts, debug scripts, or get explanations. You may not use AI to write your report. The distinction is the same as using a calculator versus asking someone else to sit your exam. The report must reflect your thinking, your words, and your conclusions.